

Member 5 — Testing / Learning Experience

Make sure the tutor actually teaches well

Member 5 — Testing / Learning Experience

Make sure the tutor actually teaches well

1. Your Role

Test the full product from a student's point of view and find incorrect, confusing or broken behavior.

Your work must connect to the shared end goal: Build a simple AI Maths Tutor that helps students understand mathematics, not only get answers. It uses simple English, analogies, step-by-step maths, word-based visualizations, RAG, chat history, personalized explanation styles, and interactive practice.

2. Prompt to Begin Your Work

You are Member 5 of a 5-person AI Maths Tutor project. Your responsibility is Testing / Learning Experience.

Project goal:
Make sure the complete system is correct, understandable and useful for students.

Your work:

- Create maths test questions.
- Test AI answer quality.
- Test RAG retrieval and grounding.
- Test chat history and follow-ups.
- Test explanation styles.
- Test UI.
- Test complete end-to-end flows.
- Record bugs clearly.
- Verify fixes.

Important:
Do not only test whether the API returns 200. Test whether the answer teaches correctly and clearly.

Start by producing:

1. Test strategy
2. Maths question bank
3. AI evaluation checklist
4. RAG evaluation checklist
5. Chat-history test cases
6. UI test cases
7. End-to-end scenarios
8. Bug-report format
9. Definition-of-done checklist

3. Your Roadmap

Phase	Focus	What you do	Output
Phase 1	Test planning	Define what correct, clear and useful means.	Test strategy
Phase 2	AI tests	Create concept, numerical, mistake and style tests.	AI test suite
Phase 3	RAG tests	Verify retrieval and grounding.	RAG test suite
Phase 4	Backend tests	Test auth, chat and history APIs.	Backend QA
Phase 5	Frontend tests	Test screens, rendering and error states.	UI QA

Stage	Name	What you do	Output
Phase 6	Learning tests	Check whether explanations help students understand.	Learning review
Phase 7	End-to-end	Run full student journeys.	E2E report
Phase 8	Final QA	Verify fixes and Definition of Done.	Final QA sign-off

4. What You Own

- Test strategy
- Maths question bank
- AI quality checks
- RAG checks
- Backend/UI checks
- End-to-end QA
- Bug tracking

5. Dependencies and Handoffs

The project is connected. Do not work in isolation.

From	To	Handoff
Member 2	Member 1 + 3	Retrieved context and metadata format
Member 1	Member 3	AI request/response contract and tutor behavior
Member 3	Member 4	API endpoints and JSON responses
Member 4	Member 3	Frontend API requirements and UI behavior
Member 5	All	Test cases, bugs and quality feedback

6. Your Testing Checklist

- Concept correctness
- Calculation correctness
- Explanation clarity
- Analogy usefulness
- Student mistake handling
- Follow-up context
- RAG grounding
- Auth
- History
- UI
- Full flow

7. Expected Final Deliverables

- Master test suite
- Evaluation criteria

- Bug reports
- Regression checklist
- End-to-end report
- Final Definition-of-Done checklist

8. Connection to the Final Product

The final student journey is:

Student logs in

- opens/creates a conversation
- asks a maths question
- backend receives the question + useful chat context
- RAG finds relevant approved maths content when needed
- AI creates a simple teaching response
- frontend displays the response clearly
- message is stored
- student asks a follow-up
- the system understands the context
- student can change explanation style
- student can practice and receive feedback.

Your work is complete only when it supports this shared journey, not merely when your individual code runs.

9. Definition of Done for This Member

- Your main modules/tasks are working.
- Your work has been tested with realistic examples.
- Your interfaces/contracts are documented.
- Your output can be used by the next dependent member.
- Known limitations are written down.
- Your work has been integrated into the shared project.
- Member 5 can test your feature without needing undocumented steps.

10. Important Team Rule

Keep the project simple. Do not introduce Kubernetes, microservices, complex cloud architecture, unnecessary AI agents, multiple databases, or a deployment pipeline. The current goal is a clean, understandable and functional student project.